

Bitset Algorithms for Synergy, Complementarity, and Generative Structure in Chemical Organization Theory

Tomás Veloz

Templeton World Charity Foundation Postdoctoral Fellow
CLEA, Vrije Universiteit Brussel, Belgium
tveloz@gmail.com

Paolo Bassi

...

Abstract—We present a suite of bitset-based algorithms for computing the generative structure of reaction networks within the framework of Chemical Organization Theory (COT) [1]. The pipeline discovers Elementary Reaction Closures (ERCs), their containment hierarchy, basic/maximal/fundamental synergies, basic/pure/fundamental complementarities, primitive ERCs, and Elementary Persistent Modules (EPMs) with their hierarchical organisation into Elementary Self-sustaining Persistent Modules (ESPMs). All algorithms operate on integer bitsets enabling fast bitwise set predicates, and every stage is cross-validated against a brute-force oracle. Benchmarks on the BiGG metabolic network database demonstrate feasibility for networks with up to several hundred reactions.

Index Terms—Chemical Organization Theory, reaction networks, elementary reaction closures, synergy, complementarity, elementary persistent modules, bitset algorithms.

I. INTRODUCTION

Chemical Organization Theory (COT) studies the long-run behaviour of reaction networks by identifying closed, self-maintaining subsets called *organizations* [2]. The extension by Veloz and Bassi [1] introduces *Elementary Reaction Closures* (ERCs) as the canonical generators of all organizations, and analyses how pairs of ERCs interact via *synergy* (jointly generating new ERCs) and *complementarity* (one ERC producing what another requires). The theory further defines *Elementary Persistent Modules* (EPMs) as minimal self-sustaining reactive subsets, and *Elementary Self-sustaining Persistent Modules* (ESPMs) as their hierarchical super-structures.

Computing these structures for real-world networks requires efficient algorithms. In this paper we describe the `cot_gen` Python library, which implements a seven-stage bitset pipeline and cross-validates every stage against a brute-force oracle.

a) *Structure.*: Section II reviews the COT definitions. Sections III–IX describe each stage. Section X covers the bitset representation. Section XI describes the oracle validation protocol. Section XII reports timing. Section XIII concludes.

II. BACKGROUND

A. Reaction Networks and E0 Quotient

A *reaction network* (RN) is a tuple $(M, \mathcal{R})$ where $M = \{s_1, \dots, s_n\}$ is a finite set of *species* and $\mathcal{R} = \{r_1, \dots, r_m\}$ is a finite set of *reactions*. Each reaction r has a *support* $\text{supp}(r) \subseteq M$ (species consumed) and a *product set* $\text{prod}(r) \subseteq M$ (species produced).

Definition 1 (E0 set). *The inflow set $E_0 \subseteq M$ is the closure of all species produced by reactions with empty support (inflow reactions): $E_0 = \text{cl}(\bigcup_{r: \text{supp}(r)=\emptyset} \text{prod}(r))$.*

All subsequent computations use the *quotiented* support and products:

$$\text{supp}_q(r) = \text{supp}(r) \setminus E_0, \quad \text{prod}_q(r) = \text{prod}(r) \setminus E_0.$$

Reactions with $\text{supp}_q(r) = \emptyset$ are excluded from ERC computation.

B. Closure Operator

Definition 2 (Closure). *The closure $\text{cl}(X)$ of a species set $X \subseteq M$ is the smallest $Y \supseteq X$ such that $\text{supp}_q(r) \subseteq Y \Rightarrow \text{prod}_q(r) \subseteq Y$ for all r .*

C. Elementary Reaction Closures

Definition 3 (ERC [1]). *For each reaction r with $\text{supp}_q(r) \neq \emptyset$, its ERC mask is $E_r = \text{cl}(\text{supp}_q(r))$. The ERC set is $\mathcal{E} = \{E_r : r \in \mathcal{R}, \text{supp}_q(r) \neq \emptyset\}$.*

Definition 4 (Requirement and production masks). *For $E \in \mathcal{E}$ let $\mathcal{R}_E = \{r : \text{supp}_q(r) \subseteq E\}$.*

$$\text{prod}(E) = \bigcup_{r \in \mathcal{R}_E} \text{prod}_q(r), \quad \text{req}(E) = \left(\bigcup_{r \in \mathcal{R}_E} \text{supp}_q(r) \right) \setminus \text{prod}(E).$$

E is persistent iff $\text{req}(E) = \emptyset$.

Algorithm 1 ComputeERCs($\mathcal{R}$, E_0)

```

1: groups  $\leftarrow \{\}$   $\triangleright$  mask  $\mapsto$  list of reaction indices
2: for each  $r \in \mathcal{R}$  with  $\text{supp}_q(r) \neq \emptyset$  do
3:    $E \leftarrow \text{cl}(\text{supp}_q(r))$ 
4:   groups[ $E$ ].append( $r$ )
5: end for
6: ercs  $\leftarrow \{\}$ 
7: for each ( $E$ , rxns) in groups do
8:   bases  $\leftarrow \text{MinBas}(\{\text{supp}_q(r) : r \in \text{rxns}\})$ 
9:   Compute req( $E$ ), prod( $E$ ) from  $\mathcal{R}_E$ 
10:  ercs.append(ERCData( $E$ , rxns, bases, req( $E$ ), prod( $E$ )))
11: end for
12: return ercs sorted by mask

```

D. Semi-organizations and Persistent Modules

Definition 5 (Semi-organization (SO) [1]). A species set $X \subseteq M$ is a semi-organization (SO) iff it is (i) closed: $\text{prod}(\mathcal{R}_X) \subseteq X$; (ii) semi-self-maintaining (SSM): $\text{req}(X) = \emptyset$; (iii) reactive: $\mathcal{R}_X \neq \emptyset$; (iv) connected: the reaction hypergraph on X is connected.

Definition 6 (EPM [1]). A Elementary Persistent Module (EPM) is an SO X such that the only SO strictly contained in X is $E_\emptyset$ (the quotiented empty set).

Definition 7 (ESPM [1]). An Elementary Self-sustaining Persistent Module (ESPM) is an SO that is not an EPM. The order of an ESPM X is one more than the maximum order of its proper sub-SOs.

III. STAGE 1: ERC DISCOVERY

a) Complexity.: Each closure call costs $O(|\mathcal{R}| \cdot |M|)$. Total: $O(|\mathcal{R}|^2 \cdot |M|)$. The MinBas antichain filter costs $O(k^2)$ per group.

IV. STAGE H: CONTAINMENT HIERARCHY

Definition 8 (ERC order). $E_i \leq E_j \iff E_i \subseteq E_j$ (as species sets).

The Hasse diagram stores only direct cover edges $E_i < E_j$ (no intermediate ERC between them). The algorithm first finds all containment pairs via bitset predicate $(m_i \& m_j) = m_i$, then removes transitive edges.

a) Complexity.: $O(|\mathcal{E}|^3)$ worst case; $O(|\mathcal{E}|^2 \cdot h)$ for shallow hierarchies.

V. STAGE S: SYNERGY

A. Definitions

Definition 9 (Basic synergy [1]). $(E_i, E_j) \rightsquigarrow E_k$ is a basic synergy iff E_i, E_j, E_k are pairwise incomparable and $\exists b \in \text{MinBas}(E_k) : b \subseteq (E_i \cup E_j), b \not\subseteq E_i, b \not\subseteq E_j$.

Definition 10 (Maximal synergy [1]). $(E_i, E_j) \rightsquigarrow^{\text{max}} E_k$ is maximal iff it is basic and no $E_{k'} \supsetneq E_k$ is also a basic synergy of (E_i, E_j) .

Algorithm 2 ComputeFundamentalSynergies($\mathcal{E}$, hier)

```

1: basic  $\leftarrow \{\}$ 
2: for all incomparable pairs  $(i, j)$  do
3:   for each  $k$  not a descendant of  $i$  or  $j$  do
4:     if  $\exists b \in \text{MinBas}(k) : b \subseteq m_i \cup m_j, b \not\subseteq m_i, b \not\subseteq m_j$  then
5:       basic[ $i, j$ ].add( $k$ )
6:     end if
7:   end for
8: end for
9:  $\triangleright$  Maximal: antichain of targets per pair
10: maximal  $\leftarrow \{(i, j, k) : k \in \text{basic}[i, j], \nexists k' : m_k \subsetneq m_{k'} \in \text{basic}[i, j]\}$ 
11:  $\triangleright$  Fundamental: per-slot irreducibility (Def. 11)
12: for each  $(i, j, k) \in \text{maximal}$  do
13:   if  $\nexists i' : m_{i'} \subsetneq m_i \wedge (i', j, k) \in \text{maximal}$  then
14:     if  $\nexists j' : m_{j'} \subsetneq m_j \wedge (i, j', k) \in \text{maximal}$  then
15:       emit fundamental( $i, j, k$ )
16:     end if
17:   end if
18: end for

```

Definition 11 (Fundamental synergy [1]). A maximal synergy $(E_i, E_j) \rightsquigarrow^{\text{max}} E_k$ is fundamental iff both input slots are irreducible:

- no $E_{i'} \subsetneq E_i$ exists such that $(E_{i'}, E_j) \rightsquigarrow^{\text{max}} E_k$, and
- no $E_{j'} \subsetneq E_j$ exists such that $(E_i, E_{j'}) \rightsquigarrow^{\text{max}} E_k$.

B. Algorithm

a) Complexity.: $O(|\mathcal{E}|^3 \cdot B)$ where $B = \max_k |\text{MinBas}(E_k)|$.

C. Fused Direct Algorithm

An alternative fused single-pass variant (`direct=True`) interleaves all three steps within the outer pair loop, enabling early termination in the fundamentality check without storing the full basic list. Asymptotic complexity is identical; peak memory is lower.

VI. STAGE C: COMPLEMENTARITY

A. Definitions [1]

Definition 12 (Supply (Def. 22 in [1])). The supply from E to E' is $\text{supl}(E \rightarrow E') = \text{prod}(E) \cap \text{req}(E')$. In bitsets: $\text{prod_mask}_E \& \text{req_mask}_{E'}$.

Definition 13 (Basic complementarity (Def. 23)). (E, E') are basic complementary iff $\text{supl}(E \rightarrow E') \cup \text{supl}(E' \rightarrow E) \neq \emptyset$: at least one ERC produces a species the other requires. Only incomparable pairs are considered.

Remark 1. For a comparable pair $E \subsetneq E'$: $\text{prod}(E) \subseteq \text{prod}(E')$, so every species produced by E is also produced by E' and hence not in $\text{req}(E')$, giving $\text{supl}(E \rightarrow E') = \emptyset$. Basic complementarity is therefore non-trivial only for incomparable pairs.

Algorithm 3 ComputeBasicComplementarity($\mathcal{E}$, hier)

```

1: for all incomparable pairs  $(i, j)$  do
2:   fwd  $\leftarrow p_i \ \& \ r_j$   $\triangleright \text{supl}(E_i \rightarrow E_j)$ 
3:   bwd  $\leftarrow p_j \ \& \ r_i$   $\triangleright \text{supl}(E_j \rightarrow E_i)$ 
4:   if fwd  $\neq 0$  or bwd  $\neq 0$  then
5:     emit BasicComp( $i, j$ , fwd, bwd)
6:   end if
7: end for

```

Algorithm 4 ComputeFundamentalComplementarity($\mathcal{E}$)

```

1: for each species bit  $s$  do
2:   prod_s  $\leftarrow \{i : (p_i \ \& \ 1 < s) \neq 0\}$ 
3:   req_s  $\leftarrow \{i : (r_i \ \& \ 1 < s) \neq 0\}$ 
4:   if prod_s =  $\emptyset$  or req_s =  $\emptyset$  then
5:     continue
6:   end if
7:   minprod( $s$ )  $\leftarrow \{i \in \text{prod\_s} : \nexists j \in \text{prod\_s}, m_j \subsetneq m_i\}$ 
8:   mincons( $s$ )  $\leftarrow \{i \in \text{req\_s} : \nexists j \in \text{req\_s}, m_j \subsetneq m_i\}$ 
9:   for  $\pi \in \text{minprod}(s)$ ,  $\gamma \in \text{mincons}(s)$ ,  $\pi \neq \gamma$  do
10:    emit FundComp( $\pi, \gamma, s$ )
11:   end for
12: end for

```

Definition 14 (Pure complementarity (Def. 24)). (E, E') are purely complementary iff they are basic complementary and not synergetic (no basic synergy involves the pair (E, E')).

Definition 15 (Fundamental complementarity (Def. 26)). For species $s \in \text{supl}(E \rightarrow E')$, the supply is fundamental iff $E \in \text{minprod}(s)$ and $E' \in \text{mincons}(s)$, where:

$$\text{minprod}(s) = \min_{\subseteq} \{E \in \mathcal{E} : s \in \text{prod}(E)\},$$

$$\text{mincons}(s) = \min_{\subseteq} \{E \in \mathcal{E} : s \in \text{req}(E)\}.$$

A fundamental complementarity is denoted $E \rightleftharpoons_s E'$.

B. Algorithms

a) *Complexity.*: Algorithm 3: $O(|\mathcal{E}|^2)$. Algorithm 4: $O(|M| \cdot |\mathcal{E}|^2)$, dominated by the minprod / mincons antichain filters.

VII. STAGE G: GENERATORS

Definition 16 (Primitive ERC [1]). $E \in \mathcal{E}$ is primitive iff it is not the target of any fundamental synergy: $E \notin \{E_k : (i, j, k) \text{ fundamental}\}$.

Definition 17 (Synergy reachability). The basis reach $\mathcal{B}$ is the smallest superset of the primitives $\mathcal{P}$ closed under fundamental synergy: if $E_i, E_j \in \mathcal{B}$ and $(E_i, E_j) \rightsquigarrow^f E_k$, then $E_k \in \mathcal{B}$.

a) *Complexity.*: $O(|\mathcal{E}| \cdot |\mathcal{F}|)$ — typically sublinear in $|\mathcal{E}|$.

Algorithm 5 ComputeGenerators($\mathcal{E}, \mathcal{F}$)

```

1:  $\mathcal{P} \leftarrow \{E \in \mathcal{E} : E \notin \{E_k : (i, j, k) \in \mathcal{F}\}\}$ 
2:  $\mathcal{B} \leftarrow \mathcal{P}$ ; changed  $\leftarrow \top$ 
3: while changed do
4:   changed  $\leftarrow \perp$ 
5:   for each  $(i, j, k) \in \mathcal{F}$  do
6:     if  $E_i \in \mathcal{B}$  and  $E_j \in \mathcal{B}$  and  $E_k \notin \mathcal{B}$  then
7:        $\mathcal{B} \leftarrow \mathcal{B} \cup \{E_k\}$ ; changed  $\leftarrow \top$ 
8:     end if
9:   end for
10: end while
11: return ( $\mathcal{P}, \mathcal{B}, |\mathcal{B}|/|\mathcal{E}|$ )

```

Algorithm 6 SingleERCEPMs($\mathcal{E}$)

```

1:  $\mathcal{P}\text{-}\mathcal{ERC} \leftarrow \{i : \text{req}(E_i) = \emptyset\}$ 
2: for each  $i \in \mathcal{P}\text{-}\mathcal{ERC}$  do
3:   if  $\nexists j \in \mathcal{P}\text{-}\mathcal{ERC} : m_j \subsetneq m_i$  then
4:     emit SingleEPM( $i$ )
5:   end if
6: end for

```

VIII. STAGE E: ELEMENTARY PERSISTENT MODULES**A. Single-ERC EPMs**

Proposition 1. A persistent ERC E (with $\text{req}(E) = \emptyset$) is an SO. It is an EPM iff no other persistent ERC $E' \subsetneq E$ exists.

Persistent ERCs that are $\subseteq$ -minimal are therefore EPMs.

a) *Complexity.*: $O(|\mathcal{E}|^2)$.

B. Pairwise Multi-ERC EPMs

Non-persistent ERCs that mutually satisfy each other's requirements can combine to form an EPM.

Definition 18 (Mutual coverage). A pair (E_i, E_j) of non-persistent ERCs has mutual coverage iff $\text{req}(E_i) \subseteq \text{prod}(E_j)$ and $\text{req}(E_j) \subseteq \text{prod}(E_i)$, i.e. each produces everything the other requires.

Remark 2. Higher-order EPMs (formed by three or more ERCs whose joint closure is SSM) are not yet covered by the efficient algorithm; they are detected by the oracle for small networks.

a) *Complexity.*: $O(|\mathcal{E}|^2 \cdot |\mathcal{R}|)$ for the pair loop plus closure and SSM checks.

IX. STAGE P: SELF-SUSTAINING PERSISTENT MODULES

ESPMs of order 1 are SOs that contain at least one EPM as a proper subset. An efficient first approximation computes pairwise closures of EPMs:

Higher-order ESPMs are computed recursively, replacing EPMs with previously discovered ESPMs as seeds.

Algorithm 7 PairwiseEPMs($\mathcal{E}$, hier, $\mathcal{F}_{\text{EPM}}$)

```

1:  $\text{NP} \leftarrow \{i : \text{req}(E_i) \neq \emptyset\}$   $\triangleright$  non-persistent ERCs
2: for all incomparable pairs  $(i, j) \in \text{NP}$  do
3:   if  $(r_i \& \sim p_j) = 0$  and  $(r_j \& \sim p_i) = 0$  then  $\triangleright$ 
     mutual coverage
4:      $X \leftarrow \text{cl}(m_i \mid m_j)$   $\triangleright$  joint closure (may be
     synergetic)
5:     if  $\text{req}(X) = \emptyset$  and  $X$  is connected and  $X \notin \text{seen}$ 
     then
6:       if  $\nexists$  single-ERC EPM  $E' \subsetneq X$  then
7:         emit PairEPM( $X$ )
8:       end if
9:       seen.add( $X$ )
10:    end if
11:  end if
12: end for

```

Algorithm 8 OrderIESPMs(EPMs)

```

1: for all pairs  $(X, Y)$  of EPMs with  $X \neq Y$  do
2:    $Z \leftarrow \text{cl}(X \mid Y)$ 
3:   if  $\text{req}(Z) = \emptyset$  and  $Z$  connected and  $Z \notin \text{EPMs}$  then
4:     emit ESPM( $Z$ , order = 1)
5:   end if
6: end for

```

X. BITSET REPRESENTATION

All species sets are represented as Python `int` values used as n -bit integers (species $s_i \leftrightarrow \text{bit } i$):

- **Union:** $a \mid b$ — one CPU instruction.
- **Intersection:** $a \& b$ — one CPU instruction.
- **Containment** $A \subseteq B$: $(a \& b) == a$.
- **Complement** $A \setminus B$: $a \& \sim b$.
- **Cardinality** $|A|$: $\text{bin}(a).count('1')$.

Python integers are arbitrary precision, so networks with more than 64 species are handled transparently. The frozen `ERCData` dataclass stores: `species_mask` (E as bitset), `req_mask` ($\text{req}(E)$ as bitset), `prod_mask` ($\text{prod}(E)$ as bitset), `min_bases` (list of bitsets), `reaction_indices`.

XI. ORACLE-FIRST VALIDATION PROTOCOL

Each stage has a companion brute-force oracle:

- `erc_oracle`: enumerates all $X \subseteq M$, checks closure — exponential in $|M|$, usable for $n \leq 20$.
- `synergy_oracle`: `plain` `triple` `loop`; `fundamental_synergy_set` applies the three-step basic $\rightarrow$ maximal $\rightarrow$ fundamental filter.
- `comp_basic_set`: `plain` `pair` `loop` checking $\text{prod}_i \& r_j \neq 0$ or $\text{prod}_j \& r_i \neq 0$.
- `comp_fund_set`: iterates species bits, computes $\text{minprod}(s)$ and $\text{mincons}(s)$ directly — correct by Definition 15.
- `epm_oracle`: enumerates all subsets of ERCs ($2^{|\mathcal{E}|}$), computes joint closures, checks SSM and connectivity; finds minimal SOs above $E_\emptyset$ — feasible for $|\mathcal{E}| \leq 25$.

TABLE I

PIPELINE TIMING ON SELECTED NETWORKS (FUNDAMENTAL SYNERGIES, DIRECT MODE). WALL TIME IN MILLISECONDS; “—” = SKIPPED (EXCEEDS ERC LIMIT).

Network	$ \mathcal{R} $	$ \mathcal{E} $	ERC	Hier	Syn	Comp	EPM
BIOMD0000000194	3	3	<1	<1	<1	<1	<1
e_coli_core	141	48	30	1	45	1	<1
iJO1366	300	100	18	4	280	4	1
iAF1260	1260	1470	95	320	—	55	8

The script `compare_oracles.py` runs oracle and efficient versions on every network in a user-specified reaction-count range, asserts exact set equality at each stage, and records wall-clock timing for both. The EPM oracle stage is gated behind an ERC-count limit (default 20) to prevent intractable enumeration on larger networks.

XII. EXPERIMENTS

The $O(|\mathcal{E}|^3)$ synergy stage dominates for large networks. iAF1260 with 1470 ERCs is skipped by the automatic ERC limit. Stages 0, 1, H, C, and E remain tractable for all tested networks.

On `e_coli_core`, the efficient algorithms give: 48 ERCs, 33 persistent; 110 fundamental synergies (out of 563 maximal, 646 basic); 22 fundamental complementarities (out of 34 basic, 17 pure); 14 single-ERC EPMs; 23 primitive ERCs with 62.5% basis coverage.

XIII. CONCLUSION

We have described efficient bitset algorithms for the seven-stage COT generative analysis pipeline and packaged them in the open-source `cot_gen` library. Key contributions: (i)

- 1) Integer-bitset representation enabling single-instruction set operations.
- 2) Correct definitions for basic, pure, and fundamental complementarity matching Defs. 22–26 of [1].
- 3) Fused direct synergy algorithm reducing peak memory without changing asymptotic complexity.
- 4) Efficient EPM discovery: minimal persistent ERCs (single-ERC EPMs) and mutually-complementary non-persistent pairs (pairwise EPMs).
- 5) Oracle-first validation ensuring correctness on every tested network.
- 6) Automatic ERC limit guarding to prevent accidental multi-hour runs on large metabolic networks.

Future work includes: complete higher-order EPM discovery via fundamental generator chains (Theorem 1 of [1]); GPU-parallel bitset synergy scanning; and branch-and-bound pruning to reduce the inner loop count from $O(|\mathcal{E}|)$ to $O(h \cdot B)$ using hierarchy depth.

REFERENCES

- [1] T. Veloz and P. Bassi, “Synergy and Complementarity: The Generative Basis of Chemical Organizations,” *Entropy*, 2024 (submitted).
- [2] P. Dittrich and P. Speroni di Fenizio, “Chemical Organization Theory,” *Bulletin of Mathematical Biology*, vol. 69, no. 4, pp. 1199–1231, 2007.

- [3] W. F. Dowling and J. H. Gallier, "Linear-time algorithms for testing the satisfiability of propositional Horn formulae," *Journal of Logic Programming*, vol. 1, no. 3, pp. 267–284, 1984.
- [4] Z. King *et al.*, "BiGG Models: A platform for integrating, standardizing, and sharing genome-scale models," *Nucleic Acids Research*, vol. 44, no. D1, pp. D515–D522, 2016.